BÀI 6: BẢO MẬT VỚI DEVSECOPS

Trong kỷ nguyên DevOps, tốc độ triển khai phần mềm cần song hành với bảo mật – đó là lý do DevSecOps ra đời. Bài thực hành này giúp sinh viên tích hợp bảo mật xuyên suốt trong pipeline CI/CD, từ thiết kế pipeline an toàn, kiểm thử bảo mật tự động, quét lỗ hổng container đến phân tích hạ tầng dạng mã. Qua bài tập thực hành, sinh viên sẽ xây dựng một hệ thống CI/CD có các chốt kiểm tra bảo mật tự động, từ đó nắm vững cách bảo vệ phần mềm ngay từ giai đoạn phát triển.

6.1 TÍCH HỢP BẢO MẬT TRONG CI/CD

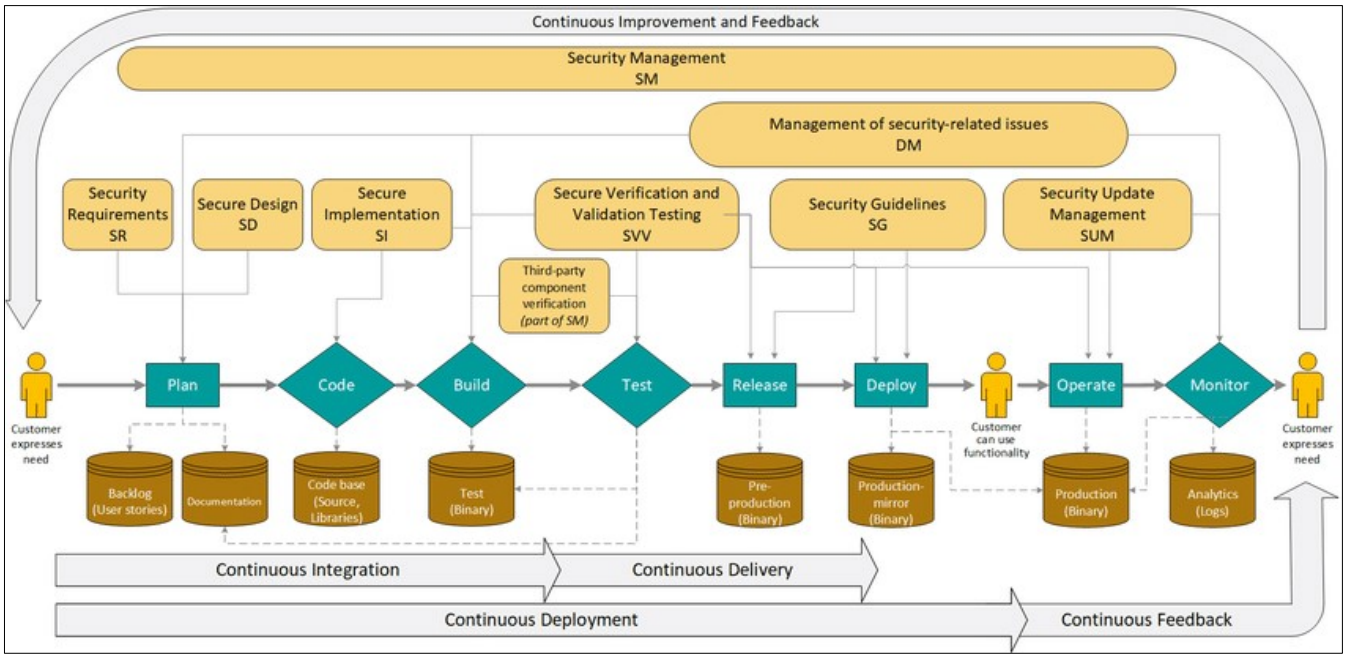
6.1.1 Thiết kế Pipeline An toàn

Thiết kế pipeline CI/CD an toàn là nền tảng cốt lõi trong DevSecOps, nhằm đảm bảo rằng mã nguồn và toàn bộ quy trình triển khai phần mềm không trở thành điểm yếu về bảo mật. Một pipeline CI/CD được xem là an toàn khi nó được thiết kế với các biện pháp bảo vệ chủ động, kiểm soát chặt chẽ và giám sát liên tục. Không chỉ dừng lại ở việc tự động hóa kiểm thử và triển khai, một pipeline bảo mật còn phải ngăn chặn được việc chèn mã độc, rò rỉ thông tin nhạy cảm, hoặc bị kẻ xấu lợi dụng hạ tầng để phát tán tấn công.

Các thành phần chính trong thiết kế pipeline an toàn:

- **Kiểm soát quyền truy cập (Access Control)**: Chỉ những tài khoản, hệ thống, hoặc dịch vụ được cấp quyền cụ thể mới có thể kích hoạt hoặc sửa đổi pipeline. Phải bật xác thực đa yếu tố (MFA) và phân quyền theo nguyên tắc "ít quyền nhất".
- **Quản lý bí mật (Secrets Management)**: Các khóa API, mật khẩu, token không được viết thẳng vào mã nguồn. Cần sử dụng kho bí mật chuyên dụng (như HashiCorp Vault, AWS Secrets Manager) và kiểm tra rò rỉ bằng các công cụ như truffleHog.

- **Ký và kiểm tra mã (Code Signing & Integrity)**: Mỗi commit và artifact phải được ký số để xác minh tính toàn vẹn. Bất kỳ sự thay đổi nào không được ký sẽ bị từ chối.
- **Tự động hóa kiểm thử bảo mật (Security Gates)**: Tích hợp các bước kiểm tra mã (SAST), phân tích phụ thuộc (Dependency Scanning), và kiểm thử động (DAST) như các "cửa chắn" bắt buộc. Nếu phát hiện lỗi bảo mật nghiêm trọng, pipeline sẽ dừng và thông báo đến developer.
- **Giám sát và audit (Monitoring & Auditing)**: Mỗi lần chạy pipeline phải được log chi tiết. Các hành vi bất thường như thay đổi pipeline, đẩy code ra môi trường production ngoài giờ hành chính cần được cảnh báo.
- **Bảo vệ CI/CD runner**: Runner là nơi thực thi pipeline – nếu bị tấn công, kẻ xấu có thể chiếm quyền điều khiển toàn bộ hệ thống. Cần cô lập runner, cập nhật thường xuyên, và hạn chế quyền truy cập mạng.



Hình 6.1: Pipeline DevOps tuân thủ tiêu chuẩn bảo mật. Nguồn: ResearchGate

6.1.2 Kiểm thử bảo mật tự động

Trong quy trình DevSecOps, kiểm thử bảo mật tự động là yếu tố then chốt nhằm phát hiện và ngăn chặn các lỗ hổng bảo mật ngay từ giai đoạn đầu của chu kỳ phát triển phần mềm. Thay vì đợi đến giai đoạn kiểm thử thủ công sau cùng, việc tích hợp

các công cụ bảo mật tự động vào pipeline CI/CD giúp phát hiện lỗi bảo mật sớm (Shift Left Security), giảm chi phí khắc phục và tăng độ an toàn tổng thể cho hệ thống. Các công cụ phổ biến bao gồm:

- **Static Application Security Testing (SAST)**: Phân tích mã nguồn để phát hiện lỗi logic, injection, hoặc lộ dữ liệu.
- **Dynamic Application Security Testing (DAST)**: Kiểm thử ứng dụng đang chạy để phát hiện các hành vi nguy hiểm trong thời gian thực.
- **Software Composition Analysis (SCA)**: Kiểm tra các thư viện bên thứ ba để phát hiện các package có lỗ hổng.
- **Secret Scanning**: Tự động dò tìm API keys, mật khẩu, token bị lộ trong mã nguồn.

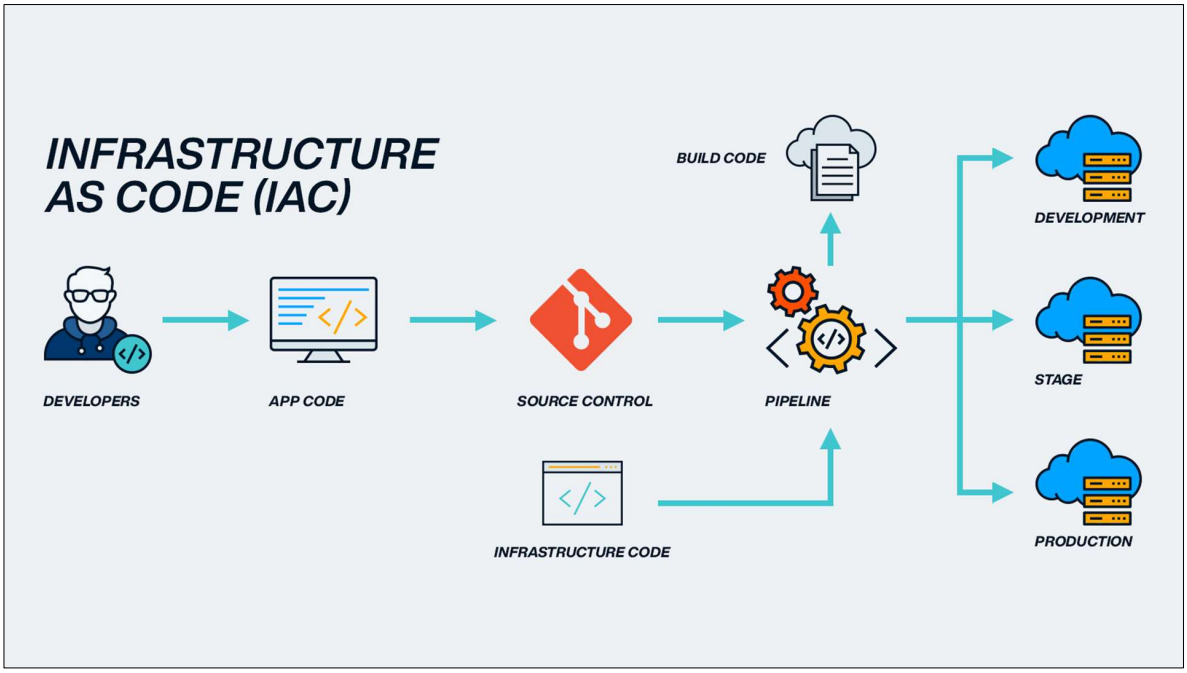
Việc triển khai kiểm thử bảo mật tự động đòi hỏi cấu hình hợp lý để tránh tạo ra quá nhiều false positives, và cần có quy trình xử lý kết quả rõ ràng. Khi thực hiện đúng cách, automated security testing trở thành lớp phòng vệ vững chắc, góp phần xây dựng pipeline DevOps không chỉ nhanh mà còn an toàn và đáng tin cậy.

6.1.3 Bảo mật Hạ tầng dưới dạng mã

Infrastructure as Code (IaC) cho phép quản lý và triển khai hạ tầng bằng mã nguồn, giúp tự động hóa và tăng tốc quá trình vận hành hệ thống. Tuy nhiên, IaC cũng tiềm ẩn nguy cơ nếu các cấu hình không an toàn hoặc có lỗ hổng thì sẽ được tái sử dụng nhanh chóng, gây ra rủi ro bảo mật nghiêm trọng. Bảo mật IaC tập trung vào việc:

- **Kiểm tra cấu hình IaC**: Phân tích các file cấu hình (như Terraform, Ansible, CloudFormation) để phát hiện các thiết lập không an toàn như quyền truy cập quá rộng, mã hóa yếu, hoặc mở cổng không cần thiết.
- **Phát hiện lỗ hổng bảo mật sớm**: Giúp ngăn chặn việc triển khai hạ tầng có nguy cơ ngay từ đầu, trước khi ảnh hưởng đến môi trường thực tế.
- **Tự động hóa bảo mật**: Tích hợp các công cụ quét bảo mật vào pipeline CI/CD để đảm bảo mọi thay đổi IaC đều được kiểm tra kỹ càng.

Việc quản lý và bảo vệ IaC giúp tạo ra môi trường vận hành vừa nhanh chóng, linh hoạt, lại vừa vững chắc trước các mối đe dọa, đảm bảo tính toàn vẹn và an toàn của hệ thống từ tầng hạ tầng đến ứng dụng.



Hình 6.2: IaC Workflow. Nguồn: SparkFabrik Blog

6.2 THỰC HÀNH CI/CD VỚI GITLAB

CI/CD là phương pháp phát triển phần mềm liên tục, nơi bạn tự động xây dựng, kiểm thử, triển khai và giám sát từng thay đổi mã nguồn theo từng bước lặp. Cách làm này giảm thiểu rủi ro do lỗi hoặc bản build trước đó chưa ổn định. GitLab CI/CD giúp phát hiện lỗi sớm và đảm bảo mã đưa vào production tuân thủ tiêu chuẩn chất lượng bạn đã đặt ra.



Hình 6.3: Gitlab CI/CD. Nguồn: Gitlab

Bước 1: Tạo tệp .gitlab-ci.yml

Để sử dụng GitLab CI/CD, bạn bắt đầu bằng tệp `.gitlab-ci.yml` ở gốc dự án của mình. Tệp này chỉ định các giai đoạn, công việc và tập lệnh sẽ được thực thi trong suốt quy trình CI/CD của bạn. Đây là tệp YAML có cú pháp tùy chỉnh riêng.

Trong tệp này, bạn định nghĩa các biến, sự phụ thuộc giữa các công việc và chỉ định thời điểm và cách thực thi từng công việc.

Bạn có thể đặt tên cho tệp này theo bất kỳ tên nào bạn muốn, nhưng `.gitlab-ci.yml` là tên phổ biến nhất và tài liệu sản phẩm gọi tệp này là tệp `.gitlab-ci.yml` hoặc tệp cấu hình CI/CD.

Bước 2: Tìm hoặc tạo runner

Runner là các tác nhân chạy các tác vụ của bạn. Các tác nhân này có thể chạy trên máy vật lý hoặc phiên bản ảo. Trong tệp .gitlab-ci.yml của bạn, bạn có thể chỉ định một hình ảnh container mà bạn muốn sử dụng khi chạy tác vụ. Runner tải hình ảnh, sao chép dự án của bạn và chạy tác vụ cục bộ hoặc trong container.

Nếu bạn sử dụng GitLab.com, các runner trên Linux, Windows và macOS đã có sẵn để sử dụng. Và bạn có thể đăng ký các runner của riêng mình trên GitLab.com nếu muốn. Nếu bạn không sử dụng GitLab.com, bạn có thể:

- Đăng ký các runner hoặc sử dụng các runner đã đăng ký cho phiên bản GitLab Self-Managed của bạn.
- Tạo một runner trên máy cục bộ của bạn.

Bước 3: Xác định pipeline của bạn

Pipeline là thứ bạn định nghĩa trong tệp `.gitlab-ci.yml` và là thứ xảy ra khi nội dung của tệp được chạy trên một trình chạy. Pipeline gồm các công việc và giai đoạn:

- Giai đoạn xác định thứ tự thực hiện. Các giai đoạn điển hình có thể là xây dựng, thử nghiệm và triển khai.
- Công việc chỉ định các tác vụ sẽ được thực hiện trong từng giai đoạn. Ví dụ: một công việc có thể biên dịch hoặc thử nghiệm mã.

Pipeline có thể được kích hoạt bởi nhiều sự kiện khác nhau, như cam kết hoặc hợp nhất, hoặc có thể theo lịch trình. Trong pipeline của bạn, bạn có thể tích hợp với nhiều công cụ và nền tảng khác nhau.

6.2.1 Tạo và chạy CI/CD pipeline

Điều kiện tiên quyết:

Trước khi bắt đầu, hãy đảm bảo bạn có:

- Một dự án trong GitLab mà bạn muốn sử dụng CI/CD.
- Vai trò Người bảo trì hoặc Chủ sở hữu cho dự án.

Nếu bạn không có dự án, bạn có thể tạo một dự án công khai miễn phí trên https://gitlab.com.

Các bước:

Để tạo và chạy pipeline đầu tiên của bạn:

- Đảm bảo bạn có sẵn các runner để chạy các tác vụ của mình.
- Nếu bạn đang sử dụng GitLab.com, bạn có thể bỏ qua bước này. GitLab.com cung cấp các runner cho bạn.
- Tạo tệp `.gitlab-ci.yml` tại thư mục gốc của kho lưu trữ. Tệp này là nơi bạn xác định các tác vụ CI/CD.

Khi bạn commit tệp vào kho lưu trữ của mình, runner sẽ chạy các tác vụ của bạn. Kết quả tác vụ được hiển thị trong một pipeline.

Đảm bảo bạn có sẵn các trình chạy:

Trong GitLab, trình chạy là các tác nhân chạy các tác vụ CI/CD của bạn.

Nếu bạn đang sử dụng GitLab.com, bạn có thể bỏ qua bước này. GitLab.com cung cấp trình chạy phiên bản cho bạn. Để xem các trình chạy khả dụng:

- Trên thanh bên trái, chọn Tìm kiếm hoặc đi đến và tìm dự án của bạn.
- Chọn Cài đặt > CI/CD.
- Mở rộng Runner.

Miễn là bạn có ít nhất một trình chạy đang hoạt động, có vòng tròn màu xanh lá cây bên cạnh, bạn sẽ có một trình chạy khả dụng để xử lý các tác vụ của mình. Nếu bạn không có quyền truy cập vào các cài đặt này, hãy liên hệ với quản trị viên GitLab của bạn.

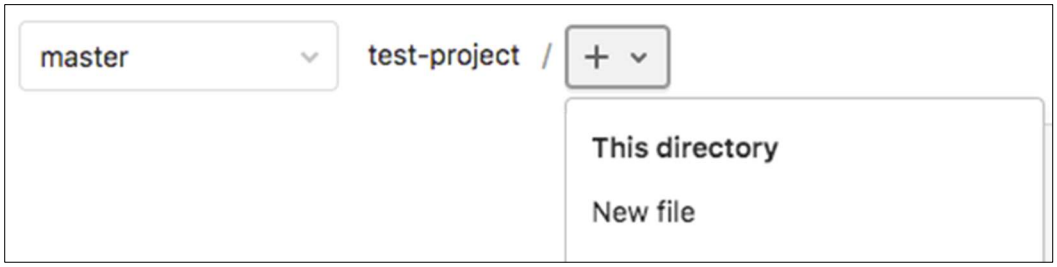
Tạo tệp `.gitlab-ci.yml`

Bây giờ hãy tạo tệp `.gitlab-ci.yml`. Đây là tệp YAML, trong đó bạn chỉ định hướng dẫn cho GitLab CI/CD. Trong tệp này, bạn định nghĩa:

- Cấu trúc và thứ tự các công việc mà trình chạy phải thực thi.
- Các quyết định mà trình chạy phải đưa ra khi gặp phải các điều kiện cụ thể.

Để tạo tệp `.gitlab-ci.yml` trong dự án của bạn:

- Trên thanh bên trái, hãy chọn Tìm kiếm hoặc đi đến và tìm dự án của bạn.
- Chọn Mã > Kho lưu trữ.
- Phía trên danh sách tệp, hãy chọn nhánh bạn muốn cam kết. Nếu bạn không chắc chắn, hãy để nguyên master hoặc main. Sau đó, chọn biểu tượng dấu cộng (+) và Tệp mới:



- Đối với Tên tệp, nhập `.gitlab-ci.yml` và trong cửa sổ lớn hơn, dán code mẫu này:

```
build-job:
  stage: build
  script:
    - echo "Hello, $GITLAB_USER_LOGIN!"
test-job1:
  stage: test
  script:
    - echo "This job tests something"
test-job2:
  stage: test
  script:
    - echo "This job tests something, but takes more time than test-job1."
```

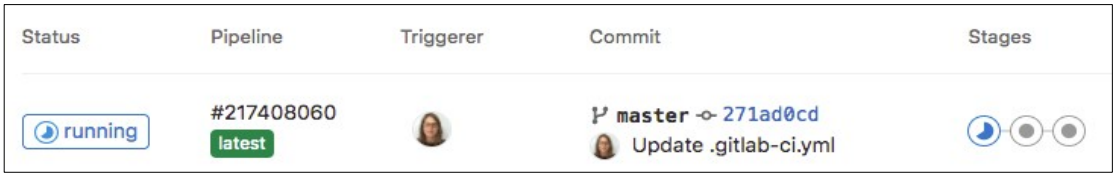
```
    - echo "After the echo commands complete, it runs the sleep command for 20 seconds"
    - echo "which simulates a test that runs 20 seconds longer than test-job1"
    - sleep 20
deploy-prod:
  stage: deploy
  script:
    - echo "This job deploys something from the $CI_COMMIT_BRANCH branch."
  environment: production
```

- Ví dụ này hiển thị bốn công việc: `build-job`, `test-job1`, `test-job2` và `deploy-prod`. Các bình luận được liệt kê trong lệnh echo được hiển thị trong UI khi bạn xem các công việc.
- Các giá trị cho các biến được xác định trước `$GITLAB_USER_LOGIN` và `$CI_COMMIT_BRANCH` được điền khi các công việc chạy.
- Chọn Commit thay đổi. Pipiline bắt đầu và chạy các công việc bạn đã xác định trong tệp `.gitlab-ci.yml`.

Xem trạng thái của pipeline và công việc của bạn

Bây giờ hãy xem pipelinevà công việc bên trong.

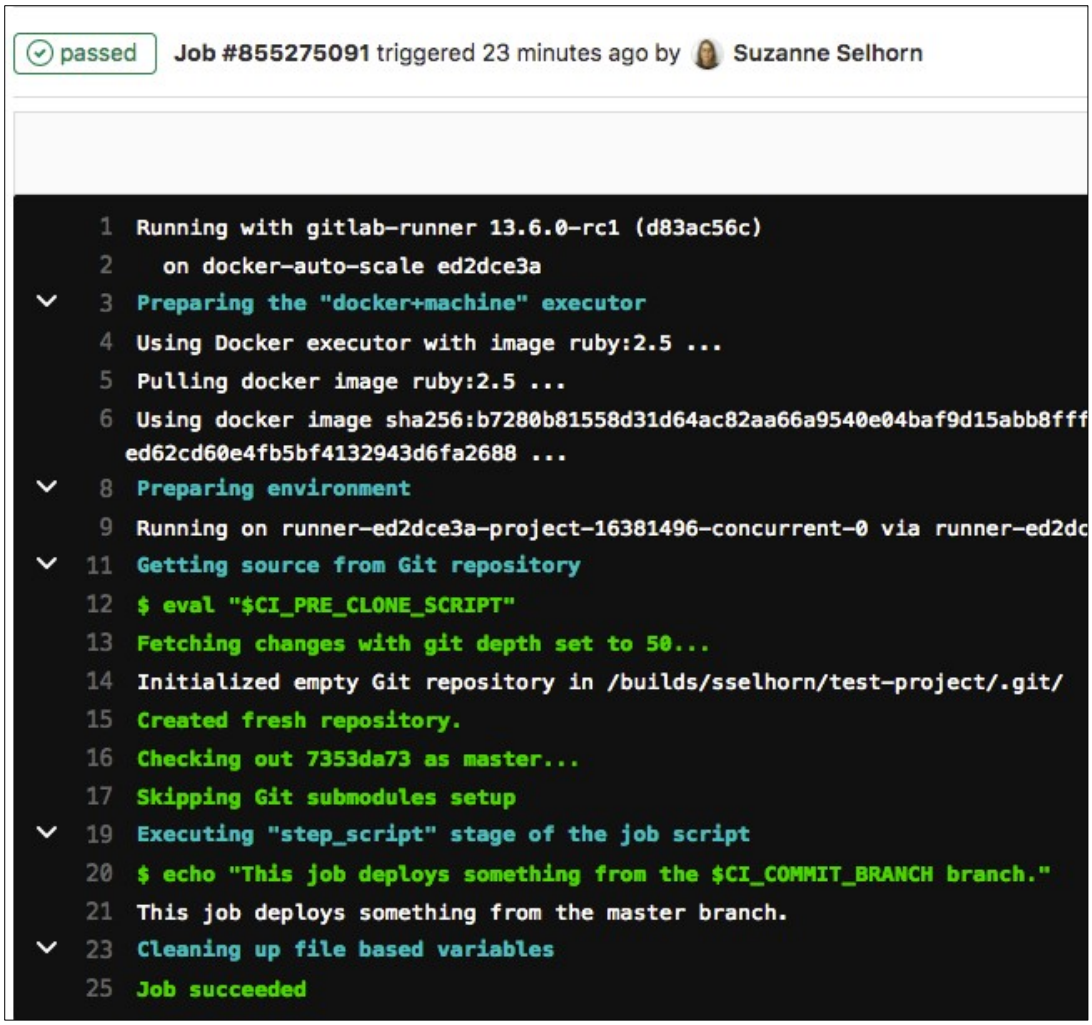
- Đi đến Build>Pipeline. Pipelinecó ba giai đoạn sẽ được hiển thị:



Xem hình ảnh trực quan về pipelinecủa bạn bằng cách chọn ID pipeline:



Xem chi tiết của một công việc bằng cách chọn tên công việc. Ví dụ `deploy-prod`:



Bạn đã tạo thành công pipiline CI/CD đầu tiên của mình trong GitLab. Xin chúc mừng! Bây giờ bạn có thể bắt đầu tùy chỉnh `.gitlab-ci.yml` của mình và xác định các công việc nâng cao hơn.

6.2.2 Tạo pipeline CI/CD phức tạp

Yêu cầu

- Bạn cần có một tài khoản trên GitLab.com.
- Bạn nên quen thuộc với Git.
- Node.js phải được cài đặt trên máy cục bộ của bạn. Ví dụ, trên macOS, bạn có thể cài đặt node bằng lệnh `brew install node`.

Tạo dự án Docusaurus

Trước khi thêm cấu hình pipeline, trước tiên bạn phải thiết lập một dự án Docusaurus trên GitLab.com:

- Tạo một dự án mới dưới tên người dùng của bạn (không phải nhóm):
  - Trên thanh bên trái, ở trên cùng, chọn Tạo mới (+) và Dự án/kho lưu trữ mới. Chọn Tạo dự án trống.
  - Nhập thông tin chi tiết về dự án: Trong trường Tên dự án, nhập tên dự án của bạn, ví dụ My Pipeline Tutorial Project. Chọn Khởi tạo kho lưu trữ bằng tệp README. Chọn Tạo dự án.
- Trên trang tổng quan của dự án, ở góc trên bên phải, hãy chọn Mã để tìm đường dẫn sao chép cho dự án của bạn.
- Sao chép đường dẫn SSH hoặc HTTP và sử dụng đường dẫn đó để sao chép dự án cục bộ.

```
git clone git@gitlab.com:my-username/my-pipeline-tutorial-project.git pipeline-tutorial
```

Chuyển đến thư mục của dự án, sau đó tạo một trang Docusaurus mới:

```
cd pipeline-tutorial

npm init docusaurus
```

Trình hướng dẫn khởi tạo Docusaurus sẽ nhắc bạn những câu hỏi về trang web. Sử dụng tất cả các tùy chọn mặc định. Trình hướng dẫn khởi tạo thiết lập trang web trong `website/`, nhưng trang web phải nằm trong thư mục gốc của dự án. Di chuyển các tệp lên thư mục gốc và xóa thư mục cũ:

```
mv website/* .

rm -r website
```

Cập nhật tệp cấu hình Docusaurus với các chi tiết về dự án GitLab của bạn. Trong docusaurus.config.js:

- Đặt `url`: thành đường dẫn có định dạng này: `https://<my-username>.gitlab.io/`.

- Đặt `baseUrl`: thành tên dự án của bạn, như `/my-pipeline-tutorial-project/`.

Xác nhận những thay đổi và đẩy chúng lên GitLab:

```
git add .

git commit -m "Add simple generated Docusaurus site"

git push origin
```

Tạo tệp cấu hình CI/CD ban đầu

Bắt đầu với tệp cấu hình pipeline đơn giản nhất để đảm bảo CI/CD được kích hoạt trong dự án và có các runner sẵn sàng chạy các job.

Bước này giới thiệu:

- **Jobs**: Đây là các phần độc lập trong một pipeline, chịu trách nhiệm chạy các lệnh của bạn. Các job chạy trên các runner tách biệt với máy chủ GitLab.
- **script**: Phần này trong cấu hình job định nghĩa các lệnh sẽ thực thi. Nếu có nhiều lệnh (trong một mảng), chúng sẽ chạy tuần tự. Mỗi lệnh được thực thi như khi chạy từ dòng lệnh (CLI). Mặc định, nếu một lệnh thất bại hoặc trả về lỗi, job sẽ được đánh dấu là thất bại và các lệnh sau đó sẽ không chạy.

Trong bước này, hãy tạo một tệp `.gitlab-ci.yml` ở thư mục gốc của dự án với cấu hình sau:

```
test-job:

  script:

    - echo "This is my first job!"

    - date
```

Commit và đẩy thay đổi này lên GitLab, sau đó:

- Truy cập Build > Pipelines và đảm bảo một pipeline chạy với job duy nhất này.
- Chọn pipeline, sau đó chọn job để xem log và kiểm tra thông báo "This is my first job!" cùng với ngày giờ hiện tại.

Bây giờ, khi đã có tệp `.gitlab-ci.yml` trong dự án, bạn có thể chỉnh sửa cấu hình pipeline sau này bằng trình chỉnh sửa pipeline.

Thêm job để build trang web

Một nhiệm vụ phổ biến của pipeline CI/CD là build mã nguồn trong dự án và triển khai nó. Hãy bắt đầu bằng cách thêm một job để build trang web.

Bước này giới thiệu:

- `image`: Chỉ định Docker container mà runner sẽ sử dụng để chạy job. Runner sẽ:
  - Tải image container và khởi chạy nó.
  - Clone dự án GitLab vào container đang chạy.
  - Thực thi các lệnh trong script từng cái một.
- `artifacts`: Các job hoạt động độc lập và không chia sẻ tài nguyên với nhau. Nếu bạn muốn sử dụng các file được tạo ra từ một job trong job khác, bạn phải lưu chúng dưới dạng artifacts trước. Sau đó, các job tiếp theo có thể tải artifacts và sử dụng các file đã được build.

Trong bước này, hãy thay thế `test-job` bằng `build-job`:

- Sử dụng image để cấu hình job chạy với image Node.js mới nhất. Docusaurus là một dự án Node.js, và image node đã tích hợp sẵn các lệnh `npm` cần thiết.
- Chạy `npm install` để cài đặt Docusaurus vào container node, sau đó chạy `npm run build` để build trang web.
- Docusaurus lưu trang web đã build trong thư mục `build/`, vì vậy hãy lưu các file này bằng `artifacts`.

```
build-job:
  image: node
  script:
    - npm install
    - npm run build
  artifacts:
    paths:
      - "build/"
```

Sử dụng Pipeline Editor để commit cấu hình pipeline này vào nhánh mặc định, sau đó kiểm tra log của job. Bạn có thể:

- Xem các lệnh npm chạy và build trang web.

- Xác nhận rằng `artifacts` đã được lưu sau khi `job` hoàn thành.
- Duyệt nội dung của file `artifacts` bằng cách chọn Browse ở bên phải log `job` sau khi `job` kết thúc.

Thêm job để triển khai trang web

Sau khi xác nhận trang Docusaurus đã được build thành công trong build-job, bạn có thể thêm một job để triển khai nó.

Bước này giới thiệu:

- `stage` và `stages`: Các cấu hình pipeline phổ biến thường nhóm các job vào các stage (giai đoạn).
  - Các job trong cùng một stage có thể chạy song song.
  - Các job ở stage sau sẽ đợi các job ở stage trước hoàn thành.
  - Nếu một job thất bại, toàn bộ stage sẽ bị coi là thất bại và các job ở stage sau sẽ không được chạy.
- GitLab Pages: Để lưu trữ trang web tĩnh của bạn, chúng ta sẽ sử dụng GitLab Pages.

Trong bước này:

- Thêm một job để tải trang web đã build và triển khai:
  - Khi sử dụng GitLab Pages, job này luôn phải có tên là pages.
  - Các `artifacts` từ `build-job` sẽ được tự động tải xuống và giải nén vào job này.
  - Tuy nhiên, GitLab Pages tìm kiếm trang web trong thư mục `public/`, vì vậy cần thêm lệnh mv để di chuyển thư mục `build/` sang `public/`.
- Thêm phần `stages` để định nghĩa thứ tự thực thi:
  - `build-job` chạy trước trong stage build.
  - `pages` chạy sau trong stage deploy.

```
stages:          # Danh sách các stage và thứ tự thực thi
  - build
```

```
  - deploy
build-job:
  stage: build   # Job này thuộc stage `build`
  image: node
  script:
    - npm install
    - npm run build
  artifacts:
    paths:
      - "build/"
pages:
  stage: deploy  # Job này thuộc stage `deploy`
  script:
    - mv build/ public/  # Di chuyển thư mục build sang public
  artifacts:
    paths:
      - "public/"
```

Kiểm tra pipeline

- Sử dụng Pipeline Editor để commit cấu hình này vào nhánh mặc định, sau đó xem chi tiết pipeline trong danh sách Pipelines. Đảm bảo rằng:
  - Hai job chạy ở hai stage khác nhau (`build` và `deploy`).
  - Sau khi job pages hoàn thành, một job `pages:deploy` sẽ xuất hiện – đây là quá trình GitLab triển khai trang Pages.
  - Khi job `pages:deploy` hoàn tất, bạn có thể truy cập trang Docusaurus mới của mình.

Xem trang web của bạn

- Trên thanh bên trái, chọn Deploy > Pages.
- Tắt tùy chọn Use unique domain (nếu có).
- Trong phần Access pages, chọn liên kết. URL sẽ có dạng:

```
https://<tên-người-dùng>.gitlab.io/<tên-dự-án>
```

TÀI LIỆU THAM KHẢO

Ayewah, N., Pugh, W., Morgenthaler, J. D., Penix, J., & Zhou, Y. (2010). Using static analysis to find bugs. *IEEE Software*.

GitHub, Inc. (n.d.). *GitHub documentation*. Retrieved June 27, 2025, from https://docs.github.com/en

GitLab. (n.d.). *GitLab CI/CD documentation*. Retrieved from https://docs.gitlab.com/

Gruhn, V. (2020). *Securing DevOps*. O'Reilly Media.

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT), RFC 7519*. Retrieved from https://tools.ietf.org/html/rfc7519

OWASP Foundation. (n.d.). *OWASP Static Code Analysis Tools*. Retrieved from https://owasp.org/www-community/Source_Code_Analysis_Tools

OWASP Foundation. (2017). *OWASP secure coding practices – Quick reference guide*. Retrieved from https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/

OWASP Foundation. (n.d.). *OWASP Top Ten*. Retrieved June 27, 2025, from https://owasp.org/www-project-top-ten/

Stallings, W. (2017). *Network security essentials: Applications and standards* (6th ed.). Pearson.

The Internet Engineering Task Force (IETF). (2008). *RFC 5280: Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile*. Retrieved from https://tools.ietf.org/html/rfc5280

The Internet Engineering Task Force (IETF). (2012). *OAuth 2.0 authorization framework, RFC 6749*. Retrieved from https://tools.ietf.org/html/rfc6749

Văn Thiên Hoàng, & Nguyễn Trọng Minh Hồng Phước. (2025). *Lập trình an ninh thông tin*. HUTECH.